COMPUTING LAB - II
ADVANCED SYSTEM SOFTWARE LAB - CSE 5261
MTech - CSE
Week - 3 Syntax Analyzer

Name : Rajat Jain
Reg No : 200913010

Lab Exercises:

1) Write a simple desk calculator in Python using recursive decent parsing, that reads an arithmetic expression, evaluates it, and then prints its numeric value. We shall build the desk calculator starting with the following grammar for arithmetic expressions:

i) E→ E+E | E – E | E * E | E/E | - E | number
The token number is a single digit between 0 and 9.

Code

```
from ply import lex
from operator import add, sub, mul, truediv, lt, gt, eq, ne, le, ge


# For the homework, uncomment the import below
# from operator import lt, gt, eq, ne, le, ge


# List of token names - required
tokens = ('LT', 'GT',
          'LTE', 'GTE',
          'EQ', 'NE',
          'AND', 'OR',
          'NOT', 'BOOL',
          'FLOAT', 'INT',
          'ADD_OP', 'MULT_OP',
          'LPAREN', 'RPAREN',
          'false', 'true')


# global variables
```

```
token = None
lexer = None
parse_error = False


# Regular expression rules for simple tokens
# r indicates a raw string in Python - less backslashes
t_ADD_OP = r'\+|-'
t_MULT_OP = r'\*|/'
t_LPAREN = r'\('
t_RPAREN = r'\)'


# Regular expression rules with some action code
# The order matters
def t_FLOAT(t):
    r'\d*\.\d+|\d+\.\d*'
    t.value = float(t.value)  # string must be converted to float
    return t


def t_INT(t):
    r'\d+'
    t.value = int(t.value)  # string must be converted to int
    return t


# For the homework, you will add a function for boolean tokens
# A string containing ignored characters (spaces and tabs)
t_ignore = ' \t'
t_true = 'TRUE'
t_false = 'FALSE'


# Lexical error handling rule
def t_error(t):
    global parse_error
    parse_error = True
    print("ILLEGAL CHARACTER: {}".format(t.value[0]))
    t.lexer.skip(1)


# For homework 2, add the comparison operators to this dictionary
```

```
SUPPORTED_OPERATORS = {'+': add, '-': sub, '*': mul, '/': truediv,
                       '<': lt, '>': gt, '==': eq, '<=': le, '>=': ge, '!=': ne}


def command():
    """
    <command> ::= <arith_expr>
    """
    result = arith_expr()
    if not parse_error:  # no parsing error
        if token:  # if there are more tokens
            error('END OF COMMAND OR OPERATOR')
        else:
            print(result)


def bool_expr():
    """
    <bool_expr> ::= <bool_term> {OR <bool_term>}
    """
    result = bool_term()
    while token and token.type == 'OR':
        operation = SUPPORTED_OPERATORS[token.value]
        match()  # match the operator
        operand = bool_term()  # evaluate the operand
        if not parse_error:
            result = operation(result, operand)
    return result


def bool_term():
    """
    <bool_term> ::= <not_factor> {AND <not_factor>}
    """
    result = not_factor()
    while token and token.type == 'AND':
        operation = SUPPORTED_OPERATORS[token.value]
        match()  # match the operator
        operand = not_factor()  # evaluate the operand
        if not parse_error:
            result = operation(result, operand)
    return result
```

```
def not_factor():
    """
    <not_factor> ::= <{NOT} <bool_factor>
    """
    result = bool_factor()


def bool_factor():
    """
    <bool_factor> BOOL | LPAREN <bool_expr> RPAREN | <comparsion>
    """
    if token and token.type == 'BOOL':
        match()
        return result
        if token and token.type == 'LPAREN':
            match()
            return bool_expr()
        else:
            error(')')
        if token and token.type == 'RPAREN':
            match()
            return result


def comparisons():
    """
    <comparison> ::= <arith_expr> [COMP_OP <arith_expr>]
    """
    result = arith_expr()
    while token and token.type == 'COMP_OP':
        operation = SUPPORTED_OPERATORS[token.value]
        match()  # match the operator
        operand = arith_expr()  # evaluate the operand
        if not parse_error:
            result = operation(result, operand)
    return eval()


def arith_expr():
    """
    <arith_expr> ::= <term> { ADD_OP <term>}
    """
```

```
    result = term()
    while token and token.type == 'ADD_OP':
        operation = SUPPORTED_OPERATORS[token.value]
        match()  # match the operator
        operand = term()  # evaluate the operand
        if not parse_error:
            result = operation(result, operand)
    return result


def term():
    """
    <term> ::= <factor> {MULT_OP <factor>}
    """
    result = factor()
    while token and token.type == 'MULT_OP':
        operation = SUPPORTED_OPERATORS[token.value]
        match()  # match the operator
        operand = factor()  # evaluate the operand
        if not parse_error:
            result = operation(result, operand)
    return result


def factor():
    """
    <factor>::= LPAREN <arith_expr> RPAREN | FLOAT | INT
    """
    if token and token.type == 'LPAREN':
        match()
        result = arith_expr()
        if token and token.type == 'RPAREN':
            match()
            return result
        else:
            error(')')
    elif token and (token.type == 'FLOAT' or token.type == 'INT'):
        result = token.value
        match()
        return result
    else:
        error('NUMBER')
```

```
def match():
    """
    Get the next token
    """
    global token
    token = lexer.token()


def error(expected):
    """
    Print an error message when an unexpected token is encountered, sets
    a global parse_error variable.
    :param expected: (string) '(' or 'NUMBER' or or ')' or anything else...
    :return: None
    """
    global parse_error
    if not parse_error:
        parse_error = True
        print('Parser error')
        print("Expected:", expected)
        if token:
            print('Got: ', token.value)
            print('line', token.lineno, 'position', token.lexpos)


def main():
    global token
    global lexer
    global parse_error
    # Build the lexer
    lexer = lex.lex()
    # Prompt for a command
    more_input = True
    while more_input:
        input_command = input("Pluto 2.0>>>")
        if input_command == 'q':
            more_input = False
            print('Bye for now')
        elif input_command:
            parse_error = False
            # Send the command to the lexer
            lexer.input(input_command)
```

```
        token = lexer.token()  # Get the first token in a global variable
        command()


if __name__ == '__main__':
    main()
```

Output

```
Enter no. of terminals: 10
Enter the terminals :
b
e
=
a
R
;
+
(
)
r
Enter no. of non terminals: 6
Enter the non terminals :
P
S
A
E
T
F
Enter the starting symbol: P
Enter no of productions: 10
Enter the productions:
P->bSe
S->AR
S->
A->a=E;
E->FT
T->+FT
T->
F->(E)
F->a
F->r
   Non Terminals          First                Follow
         P                {'b'}                {'$'}
         S              {'@', 'a'}             {'e'}
         A                {'a'}                {'R'}
         E           {'(', 'a', 'r'}        {';', ')'}
         T              {'@', '+'}          {';', ')'}
         F           {'(', 'a', 'r'}     {';', '+', ')'}
```

2) Write a python program to compute FIRST and FOLLOW for the following grammar.

P -> bSe
S -> AR | epsilon
A -> a=E;
E -> FT
T -> +FT | epsilon
F -> (E) | a | r

<u>Code</u>

```
import sys
sys.setrecursionlimit(60)


def first(string):
    first_ = set()
    if string in non_terminals:
        alternatives = productions_dict[string]

        for alternative in alternatives:
            first_2 = first(alternative)
            first_ = first_ | first_2

    elif string in terminals:
        first_ = {string}

    elif string == '' or string == '@':
        first_ = {'@'}

    else:
        first_2 = first(string[0])
        if '@' in first_2:
            i = 1
            while '@' in first_2:
                first_ = first_ | (first_2 - {'@'})
                if string[i:] in terminals:
                    first_ = first_ | {string[i:]}
                    break
                elif string[i:] == '':
                    first_ = first_ | {'@'}
                    break
```

```
            first_2 = first(string[i:])
            first_ = first_ | first_2 - {'@'}
            i += 1
        else:
            first_ = first_ | first_2
    return first_


def follow(nT):
    follow_ = set()
    prods = productions_dict.items()
    if nT == starting_symbol:
        follow_ = follow_ | {'$'}
    for nt, rhs in prods:
        for alt in rhs:
            for char in alt:
                if char == nT:
                    following_str = alt[alt.index(char) + 1:]
                    if following_str == '':
                        if nt == nT:
                            continue
                        else:
                            follow_ = follow_ | follow(nt)
                    else:
                        follow_2 = first(following_str)
                        if '@' in follow_2:
                            follow_ = follow_ | follow_2-{'@'}
                            follow_ = follow_ | follow(nt)
                        else:
                            follow_ = follow_ | follow_2
    return follow_


no_of_terminals = int(input("Enter no. of terminals: "))

terminals = []

print("Enter the terminals :")
for _ in range(no_of_terminals):
    terminals.append(input())

no_of_non_terminals = int(input("Enter no. of non terminals: "))
```

```
non_terminals = []

print("Enter the non terminals :")
for _ in range(no_of_non_terminals):
    non_terminals.append(input())

starting_symbol = input("Enter the starting symbol: ")

no_of_productions = int(input("Enter no of productions: "))

productions = []

print("Enter the productions:")
for _ in range(no_of_productions):
    productions.append(input())

productions_dict = {}

for nT in non_terminals:
    productions_dict[nT] = []

for production in productions:
    nonterm_to_prod = production.split("->")
    alternatives = nonterm_to_prod[1].split("/")
    for alternative in alternatives:
        productions_dict[nonterm_to_prod[0]].append(alternative)

FIRST = {}
FOLLOW = {}

for non_terminal in non_terminals:
    FIRST[non_terminal] = set()

for non_terminal in non_terminals:
    FOLLOW[non_terminal] = set()

for non_terminal in non_terminals:
    FIRST[non_terminal] = FIRST[non_terminal] | first(non_terminal)

FOLLOW[starting_symbol] = FOLLOW[starting_symbol] | {'$'}
for non_terminal in non_terminals:
    FOLLOW[non_terminal] = FOLLOW[non_terminal] | follow(non_terminal)
```

```
print("{: ^20}{: ^20}{: ^20}".format('Non Terminals', 'First', 'Follow'))
for non_terminal in non_terminals:
    print("{: ^20}{: ^20}{: ^20}".format(non_terminal, str(
        FIRST[non_terminal]), str(FOLLOW[non_terminal])))
```

Output

```
Simple Calculator
5+10
15
Simple Calculator
8-6
2
Simple Calculator
4*3
12
Simple Calculator
20/4
5.0
Simple Calculator
```

3) Write a python program using simple LR parser for the following grammar.
E→ E+T | T
T → T*F | F
F → (E) | digit
The token digit is a single digit between 0 and 9.

Code

```
#       SLR Parser
#       Simple LR parser implementation

# no imports ;)


"""
Constants

"""
```

```
DOT = '.'
EPSILON = ''
DOLLAR = '$'
GOTO = "goto"
SHIFT = "shift"
REDUCE = "reduce"
ACCEPT = "accept"


"""
Utility functions

"""

def attach_dot(production):
	(lhsp,rhsp) = production
	return [lhsp,DOT + rhsp]

def symbol_after_dot(item):
	(lhsi,rhsi) = item
	if rhsi.endswith(DOT):
		return EPSILON
	return rhsi.split(DOT)[1][0]

def progress_dot(item):
	(lhsi,rhsi) = item
	ls = list(rhsi)
	i = ls.index(DOT)
	ls[i],ls[i+1] = ls[i+1],ls[i]
	return [lhsi,EPSILON.join(ls)]

def remove_dot(item):
	(lhsi,rhsi) = item
	return [lhsi,rhsi.replace(DOT,EPSILON)]



"""
Construct LR(0) items

"""
```

```
def construct_items(G):

    nonterminals = set([lhs for (lhs,rhs) in G])
    terminals = set([t for (lhs,rhs) in G for t in rhs if t not in nonterminals])


    """
    closure of an item :
    if symbol after dot is non terminal
        add its productions
    repeat until no new item is added

    """
    def closure(I):
        J = list(I)
        done = False
        while not done:
            done = True
            for item in J:
                B = symbol_after_dot(item)
                if B not in nonterminals:
                    continue
                for production in G:
                    (lhsp,rhsp) = production
                    if lhsp != B:
                        continue
                    new_item = attach_dot(production)
                    if new_item in J:
                        continue
                    J.append(list(new_item))
                    done = False
        return J

    """
    goto(I,X)     :
    Transition if X occurs at state I
    creates new lr(0) item

    """
    def goto(I,X):
        J = []
        for item in I:
```

```
                B = symbol_after_dot(item)
                if B == X:
                        new_item = progress_dot(item)
                        J.append(list(new_item))
        return J

    """
    items()     :
    Get LR(0) items
    Starts from first production
    make possible gotos
    until no new item is added

    """
    def items():
        C = [ closure([ attach_dot(G[0]) ]) ]
        S = list(nonterminals.union(terminals))
        goto_moves = {}
        done = False
        while not done:
                done = True
                for index,I in enumerate(C):
                        goto_moves[index] = [None]*len(S)
                        for symbol in S:
                                J = closure(goto(I,symbol))
                                if len(J) == 0:
                                        continue
                                if J not in C:
                                        C.append(J)
                                        done = False
                                goto_moves[index][S.index(symbol)] =
C.index(J)

        return C,S,goto_moves

    return items


"""
get first and follow for non-terminals (cached)


"""
```

```
def get_first_and_follow(G):

    nonterminals = set([lhs for (lhs,rhs) in G])
    terminals = set([t for (lhs,rhs) in G for t in rhs if t not in nonterminals])
    first_cache = {}
    follow_cache = {}
    FIRST_SYMBOL = G[0][0]

    """
    first(X) :
    if first symbol in rhs
        is terminal or epsilon
            add to result
        is non terminal other than x
            add its first to result
            see next if its first contains epsilon

    """
    def first(X):
        if X in first_cache:
            return first_cache[X]
        result = set()
        X_prod = [P for P in G if P[0] == X]
        for production in X_prod:
            (lhsp,rhsp) = production
            for index,symbol in enumerate(rhsp):
                if symbol == X:
                    break
                if symbol in terminals.union(EPSILON):
                    result = result.union(symbol)
                    break
                first_y = first(symbol)
                result = result.union(first_y-{EPSILON})
                if EPSILON not in first_y:
                    break
                if index == len(rhsp)-1:
                    result = result.union({EPSILON})
        first_cache[X] = result
        return first_cache[X]

    """
    first_of(beta) :
```

same as first except beta may contain any number of symbols

```
"""
def first_of(beta):
	result = set()
	for index,symbol in enumerate(beta):
		if symbol in terminals.union({EPSILON}):
			result = result.union({symbol})
			break
		first_y = first(symbol)
		result = result.union(first_y-{EPSILON})
		if EPSILON not in first_y:
			break
		if index == len(beta)-1:
			result = result.union({EPSILON})
	return result

"""
follow(X)	:
	follow of first symbol is dollar
	for every production containing X in rhs
	find first of portion right to X
	if it is epsilon or contains epsilon
		add follow of lhs to result

"""
def follow(X):
	if X in follow_cache:
		return follow_cache[X]
	result = set()
	if X == FIRST_SYMBOL:
		result = result.union({DOLLAR})
	X_prod = [P for P in G if X in P[1]]
	for production in X_prod:
		(lhsp,rhsp) = production
		for index,symbol in enumerate(rhsp):
			if symbol != X:
				continue
			if (index==len(rhsp)-1) and (lhsp !=symbol):
				result = result.union(follow(lhsp))
				continue
			first_beta = first_of(rhsp[index+1:])
			result = result.union(first_beta-{EPSILON})
```

```
                        if (EPSILON in first_beta) and (lhsp != symbol):
                                result = result.union(follow(lhsp))


            follow_cache[X]=result
            return follow_cache[X]


        return first,follow

"""
Construct Parsing Table for G
        for transitions of FSM
            if transition symbol is terminal add shift move
            else add goto move
        for all items containing productions of type [A->alpha.]
            add reduce move to follow of lhs of given production index
            add accept move if first prouction

"""
def construct_parsing_table(G):
        items = construct_items(G)
        states,symbols,transitions = items()
        first,follow = get_first_and_follow(G)
        symbols.append(DOLLAR)
        table = [dict() for i in states]
        nonterminals = set([s for s in symbols if s.isupper()])
        terminals = set([s for s in symbols if s not in nonterminals])


        for i in range(len(states)):
            for j in range(len(symbols)-1):
                symbol = symbols[j]
                if transitions[i][j] == None:
                        continue
                if symbol in nonterminals:
                        table[i][symbol] = [GOTO,transitions[i][j]]
                else:
                        table[i][symbol] = [SHIFT,transitions[i][j]]

        for index,I in enumerate(states):
            for item in I:
                B = symbol_after_dot(item)
```

```
                if B != EPSILON:
                        continue
                (lhsi,rhsi) = item
                prod_index = G.index(remove_dot(item))
                if prod_index == 0:
                        table[index][DOLLAR] = [ACCEPT,index]
                        break
                follow_left = follow(lhsi)
                for symbol in follow_left:
                        sym_index = symbols.index(symbol)
                        if symbol in table[index]:
                                print("Conflict in state-symbol: ",index,symbol)
                                break
                        table[index][symbol] = [REDUCE,prod_index]

        return table

"""
LR Parsing Algorithm    :
        buffer contains input string with dollar at end
        state stack with top represents current state [initial = 0]
        symbol stack is shows derivation state
        find action for current [state,symbol]
        if shift then push new state and get next symbol
        if reduce
                pop given production [A->alpha]
                pop states upto length of alpha
                push goto move of new state on symbol A
        if accept
                Done :)


"""
def get_lr_parser(G):

        table = construct_parsing_table(G)


        def parser(input_string):
                buf = list(input_string)
                buf.append(DOLLAR)
                state_stack = [0]
                symbol_stack = []
```

```
        action_string = ""
        index = 0

        while True:
            symbol = buf[index]
            s = state_stack[-1]
            if symbol not in table[s]:
                action_string = "Parsing error at " + str(index) + "
symbol " + symbol
                print(action_string)
                break
            (action,value) = table[s][symbol]

            if action == SHIFT:
                action_string = "Shift " + symbol
                symbol_stack.append(symbol)
                state_stack.append(value)
                index+=1

            if action == REDUCE:
                (lhsp,rhsp) = G[value]
                for i in range(len(rhsp)):
                    state_stack.pop()
                    symbol_stack.pop()
                s = state_stack[-1]
                state_stack.append(table[s][lhsp][1])
                symbol_stack.append(lhsp)
                action_string = "Reduce " + lhsp + " -> " + rhsp

            if action == ACCEPT:
                action_string = "Accepted :)"
                print(action_string)
                break


            print(''.join(symbol_stack),'\t',action_string)

    return parser
```

```
"""
Assumptions
        1) Grammar is augmented [ first production is S-> start_symbol ]
        2) All symbols are represented by single character only
        3) Non-terminals are represented by uppercase characters

"""



"""
Simple Expression Grammar

S -> E
E -> E+T | T
T -> T*F | F
F -> (E) | x

"""
grammar = [
["S", "E"],
["E", "E+T"],
["E", "T"],
["T", "T*F"],
["T", "F"],
["F", "(E)"]]

for i in range(10):
        grammar.append(["F",f"{i}"])


"""
Any String ending with abb

S -> A
A -> aA | bA | aB
B -> bC
C -> bD
D -> epsilon

"""

"""
```

```
grammar2 = [
["S","A"],
["A","aA"],
["A","bA"],
["A","aB"],
["B","bC"],
["C","bD"],
["D",""]
]

"""

print("\nExample 1\n")
slr = get_lr_parser(grammar)
input_string = input("Enter the input string to be parsed\n")
print(f"Input string : {input_string}\n")
slr(input_string)

"""
print("\nExample 2\n")
slr = get_lr_parser(grammar2)
slr("ababb")
"""
```

<u>Output</u>

```
Example 1

Enter the input string to be parsed
3*6+2
Input string : 3*6+2

3        Shift 3
F        Reduce F -> 3
T        Reduce T -> F
T*       Shift *
T*6      Shift 6
T*F      Reduce F -> 6
T        Reduce T -> T*F
E        Reduce E -> T
E+       Shift +
E+2      Shift 2
E+F      Reduce F -> 2
E+T      Reduce T -> F
E        Reduce E -> E+T
Accepted :)
```